



Universidad Experimental De Guayana.  
Vicerrectorado Académico.  
Coordinación General De Pregrado.  
Proyecto De Carrera: Ingeniería En Informática.  
Asignatura: Lenguaje y Compiladores.



# *Análisis Léxico*

Profesor:

Félix Márquez

Alumnos:

Celimar Rojas, Cl: 31.981.398  
Victor Canchis, Cl: 27.935.276  
Blas Gonzalez, Cl: 31.074.243  
Diosdado Muñoz, Cl: 30.932.108

# ***Autómata de Pila***

Modelo computacional que extiende al autómata finito añadiendo una **memoria LIFO (pila)**. Esta memoria le permite reconocer **lenguajes independientes del contexto**, superando las limitaciones de los autómatas finitos y sentando las bases para el análisis sintáctico de lenguajes de programación.





# Componentes Principales

El modelo formal consta de **7 elementos** que definen completamente el comportamiento del autómata de pila:

$Q$  — Estados

Conjunto finito de etapas del autómata.

$\Sigma$  — Alfabeto de entrada

Símbolos permitidos en la cadena a procesar.

$\Gamma$  — Alfabeto de la pila

Símbolos almacenables en la pila.

$\delta$  — Función de transición

Define el cambio de estado según la entrada, el estado actual y la cima de la pila.

$q_0$  — Estado inicial

Punto de partida del proceso.

$Z_0$  — Símbolo inicial

Indica que la pila está vacía al inicio.

$F$  — Estados de aceptación

Determinan si la cadena analizada es válida.

# Operaciones Básicas y Tipos de AP

## Operaciones sobre la Pila

- **Push (Meter):** Inserta un símbolo en la cima de la pila.
- **Pop (Sacar):** Extrae el elemento de la cima.
- **Lectura:** Lee un símbolo de entrada y puede alterar la pila sin consumir más entrada (transiciones  $\lambda$ ).

## Tipos de Autómatas de Pila

- **Deterministas (ADP):** Tienen máximo una transición por situación; son ideales para el diseño de compiladores y analizadores sintácticos.
- **No Deterministas (ANP):** Tienen múltiples opciones de transición; son más potentes pero computacionalmente complejos.

□ A diferencia de los autómatas finitos, el no determinismo en los AP **sí aporta mayor poder de expresión**, aceptando lenguajes más complejos. En la práctica, las gramáticas de los analizadores sintácticos exigen restricciones (como LL o LR) para garantizar un procesamiento determinista.

# Ejemplos de Lenguajes Reconocidos

## 1. Paréntesis Balanceados

Lee la cadena de izquierda a derecha; apila al encontrar ( y desapila al encontrar ). Acepta la cadena si la pila queda vacía al finalizar.

## 2. El lenguaje $a^n b^n$ (donde $n \geq 0$ )

Apila las letras a una a una. Al llegar a las b, desapila una a por cada b leída. Acepta si la entrada termina exactamente cuando la pila se vacía.

## 3. Palíndromos de Longitud Impar

Apila los símbolos de la primera mitad de la palabra (w). Ignora el símbolo central c y, en la segunda mitad, compara la entrada con los elementos que desapila.



# Aplicaciones Prácticas

Las aplicaciones prácticas de los AP se centran en su capacidad para manejar jerarquías y emparejar elementos:



## Compiladores e Intérpretes

Utilizan la lógica de los AP para verificar que el código respete las Gramáticas Libres de Contexto y construir el Árbol de Sintaxis Abstracta (AST).



## Validación de HTML, XML, JSON

Verifican que las etiquetas y estructuras de un documento estén correctamente anidadas y cerradas.



## Expresiones Matemáticas y Lógicas

Comprueban el balanceo de paréntesis y transforman expresiones a Notación Polaca Inversa (Postfix) para respetar la jerarquía de operadores.



## Funciones de Editores de Código (IDEs)

Permiten realizar en tiempo real el resaltado de sintaxis, autocompletado/cierre de llaves, corchetes y paréntesis, y el plegado de bloques de código.



## Procesamiento de Lenguaje Natural (NLP)

Identifican la estructura gramatical de oraciones humanas mediante gramáticas libres de contexto, empleándose en correctores gramaticales o traductores básicos.



# Construcción de un Lexer para Dockerfiles

## ¿Qué es un Dockerfile?

Archivo de texto plano estructurado línea a línea. Cada línea sigue el formato: INSTRUCCIÓN [flags] argumentos.

## Elementos léxicos del lenguaje L

- Comentarios (inician con #)
- Instrucciones reservadas: FROM, RUN, CMD, COPY, WORKDIR, ENV, EXPOSE, entre otras
- Alias AS (usado en builds multi-etapa)
- Flags de instrucción (ej. --from, --chown)
- Variables y expansiones de entorno
- Delimitadores y operadores (=, [, ], :, /, entre otros)
- Cadenas literales entre comillas
- Identificadores (nombres de imágenes, rutas, variables)

## Del Autómata a la Expresión Regular

### Tabla de tokens (resumen — 24 tokens en total):

| Categoría  | Tokens                                                                                                                                       |
|------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| Estructura | TK_COMENTARIO, TK_INSTRUCCION, TK_AS                                                                                                         |
| Argumentos | TK_FLAG, TK_VARIABLE, TK_VARIABLE_SIMPLE                                                                                                     |
| Literales  | TK_STRING_DOBLE, TK_STRING_SIMPLE, TK_NUMERO                                                                                                 |
| Símbolos   | TK_ASIGNACION, TK_CORCHETE_ABRE/CIERRA, TK_COMA, TK_DOS_PUNTOS, TK_ARROBA, TK_BARRA, TK_PUNTO, TK_GUION, TK_PIPE, TK_AMPER, TK_INTERROGACION |
| General    | TK_IDENTIFICADOR                                                                                                                             |

# Arquitectura del Lexer y Resultados





# Arquitectura del Lexer y Resultados

El manejo de errores léxicos emplea una **estrategia de recuperación de pánico local**: registra el error, avanza un carácter y continúa, lo que permite detectar varios errores en una sola ejecución.

## Estructura del Proyecto

- tokens.py — definición de la tabla de tokens
- lexer.py — motor de tokenización
- main.py — punto de entrada / CLI

### Conclusión:

El motor de expresiones regulares de Python es suficiente para tokenizar una gramática regular no trivial como la de un Dockerfile. El mismo analizador se empaqueta como herramienta ejecutable independiente.

## Ejemplos Ejecutados

- **Ejemplo 1** (simple, válido) — 32 tokens reconocidos
- **Ejemplo 2** (multi-etapa, con variables y flags) — 102 tokens reconocidos
- **Ejemplo 3** (con errores intencionales) — 44 tokens + 2 errores léxicos detectados

## Compilación

Empaquetado con **PyInstaller** (--onefile): ejecutable **standalone**, no requiere Python instalado.

# ¿Qué es Flex?

**Flex** (Fast Lexical Analyzer Generator, 1987) es una herramienta de metacompilación escrita en C que genera analizadores léxicos en C usando expresiones regulares. Su función principal es leer el código fuente y agrupar sus caracteres en unidades lógicas con significado, llamadas **tokens** (palabras clave, símbolos, etc.), mediante un autómata finito determinístico.

1

## Definiciones

Contiene inclusiones de código C (entre `%{` y `%}`) y opciones (como `%option noyywrap` para evitar múltiples archivos).

2

## Reglas

Cada línea asocia una expresión regular con una acción en C. Ejemplos: `"fn" { printf("TOKEN: TK_FN, Lexema: %s\n", yytext); }`, identificadores `[a-zA-Z_][a-zA-Z0-9_]*`, números `[0-9]+`, operadores aritméticos `"+" | "-" | "*" | "/"`, relacionales `"==" | "!=" | "<=" | ">=" | "<" | ">"`, comentarios `//.*` y espacios en blanco `[\t\n\r]+` se ignoran con acciones vacías. Caracteres no reconocidos emiten: `printf("ERROR LEXICO: Caracter no reconocido: %s\n", yytext);`

3

## Código de Usuario

Contiene la función `main` que abre un archivo pasado por argumento y llama a `yylex()`.

# *Diseño del Lenguaje L (Subconjunto de Rust) y Proceso de Construcción*

## Componentes Soportados

- Funciones (fn), variables inmutables (let) y mutables (let mut)
- Estructuras de control: if-else, while, return
- Tipos: i32, f64, bool (true/false), char, String
- Identificadores: patrón [a-zA-Z\_][a-zA-Z0-9\_]\*
- Números: secuencias de dígitos
- Comentarios de una línea: empiezan con //; espacios se ignoran

## Proceso de Construcción del Lexer

- **Generación:** flex lexer.l genera lex.yy.c con el código C del autómata.
- **Compilación:** gcc lex.yy.c -o lexer -lfl obtiene el ejecutable.
- **Ejecución:** ./lexer programa.rs sobre un archivo de prueba.

## Ejemplo de Salida

```
TOKEN: TK_FN, Lexema: fn
TOKEN: TK_IDENTIFICADOR, Lexema: calcular_total
TOKEN: TK_PAR_ABRE, Lexema: (
TOKEN: TK_LET, Lexema: let
TOKEN: TK_MUT, Lexema: mut
...
```

Los tokens se listan secuencialmente, ignorando comentarios y espacios. El repositorio incluye lexer.l, programa.rs, el ejecutable generado y un README.md con instrucciones de instalación y uso.



# *Implementación, Compilación y Demostración*

1

Implementación del lexer en Flex (lexer.l).

2

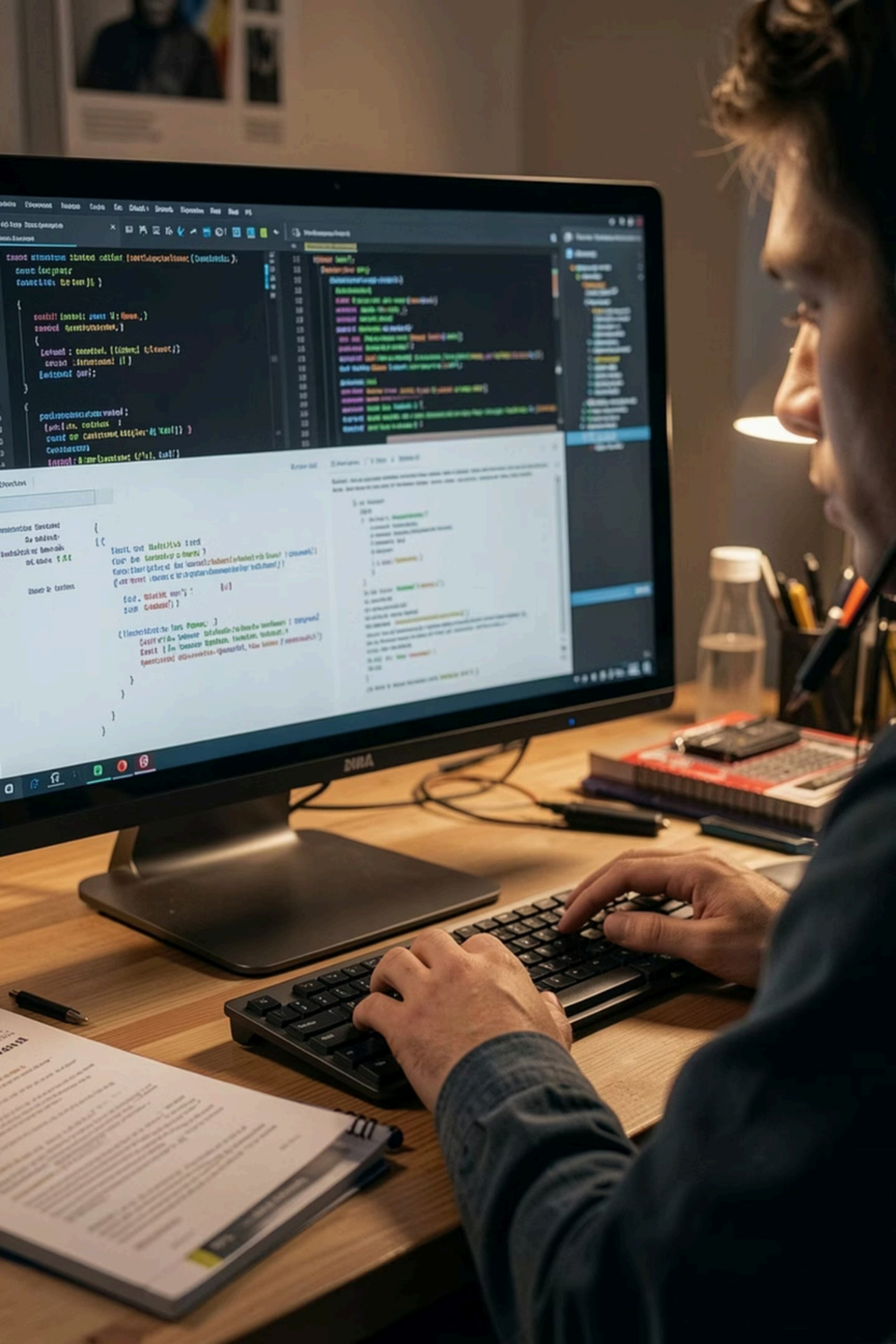
Generación automática de lex.yy.c.

3

Pruebas con archivos válidos e inválidos.

4

Reconocimiento de tokens y detección de errores léxicos.



## *Conclusión*

Flex automatiza la generación del analizador léxico.

Solo se definen las reglas mediante expresiones regulares.

El desarrollo es más rápido y fácil de mantener.

Se reducen errores de implementación frente a un autómata programado manualmente.

# *Flex en Seguridad Informática y Conclusión*

Flex analiza código fuente, tráfico de red y logs de sistemas para detectar vulnerabilidades, ataques o patrones maliciosos. Aunque genera analizadores en C/C++, puede escanear y tokenizar prácticamente cualquier lenguaje o formato de texto utilizado en ciberseguridad.

## **C / C++ y Malware**

Escanea archivos fuente buscando funciones inseguras (ej. causantes de desbordamientos de búfer). En ingeniería inversa, desensambladores tokenizan instrucciones binarias para identificar patrones de malware.

## **Firewalls y Snort / YARA**

Snort (IDS/IPS) utiliza Flex/Bison para procesar su configuración y reglas de inspección de red. YARA procesa las reglas de texto con las que los analistas describen amenazas de malware.

## **SQL y Lenguajes Web**

Parsea SQL en tiempo real dentro de un WAF para identificar inyecciones (SQL Injection). Escáneres buscan scripts maliciosos en JavaScript (XSS) o analizan PHP para encontrar puertas traseras (webshells).

## **Logs y Datos (JSON, XML, Syslog)**

Parsers ultrarrápidos que normalizan millones de líneas de texto por segundo. Buscan patrones de ataques de fuerza bruta o accesos no autorizados en eventos de seguridad.

✅ **Conclusión:** Flex automatiza la generación del analizador léxico. Solo se definen las reglas mediante expresiones regulares; el desarrollo es más rápido, fácil de mantener y se reducen errores de implementación frente a un autómata programado manualmente.



*¡Muchas Gracias  
por su atención!*

---





